

Bluesky Telemetry using Raspberry pi

Sotirios Karantzoulis

AEM: 10952

Github(<https://github.com/sot-kara/Embedded-prj>)

September 2026

1 Introduction

This project was a part of Real Time Embedded Systems class and it involves gathering data from the Bluesky social network using a raspberry pi in real time for 24 hours.

Specifically, the raspberry pi connects to a websocket from where it collects JSON files that include messages posted on the site as well as other system information. It then proceeds to categorize the messages according to their "kind" field and keeps count of each type for the duration of a second.

Due to the live nature of a social network the stream of messages from the websocket is constant and rather fast. This network characteristic demands that the application is extremely responsive while also having low CPU utilization because of the long period of data collection.

2 Architecture

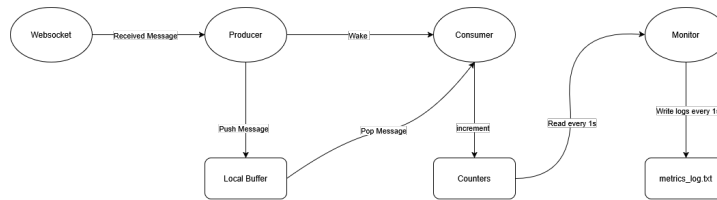


Figure 1: Overview of the System Architecture

As per the project's specifications, the producer-consumer architecture was employed so as to separate the data collection from the message categorization. This

way each part is a different thread and can work in an async manner.

In fact, the waking of the producer thread is entirely event based. It only runs when a message arrives from the websocket using a callback function. It then pushes the JSON message in a local circular buffer if it is not full (in which case it will block). The producer then signals to the consumer that there are messages awaiting categorization. In turn the consumer thread will only run when there is work to do. It will pass each message from the buffer to a JSON parser and increment the counters accordingly.

On the other hand, a separate monitor thread is used for logging. Every second the counter values as well as buffer occupancy and the CPU utilization is written to a file. As a part of the project the main goal was to minimize the jitter of this thread, so ideally it should run exactly every second.

2.1 Circular Buffer

A simple 2D char circular buffer was implemented for the purposes of queuing messages for the consumer. The basic functions of init, push, pop and were needed for the current architecture. Performance wise, it is considered better to have a static size buffer so as to avoid dynamic memory allocation, especially since this is an embedded system application.

By experimenting with the received JSON file frequency and size the buffer was made large enough to accommodate a decent quantity of long messages.

CIRC_BUFF_CAPACITY	512
MAX_JSON_LEN	4096

Table 1: Size of static circular buffer

2.2 Synchronization

The existence of shared resources like the circular buffer and the counters requires the use of parallel synchronization techniques so as to prevent data races. For this reason, a mutex is used to access the buffer (**circ_buffer_mutex**) and the counters (**counter_mutex**) whenever a thread needs to write or read these variables.

Moreover, to avoid busy waiting for the producer-consumer threads, condition variables are used. The producer blocks on the buffer being full until the consumer signals otherwise (**not_full**), and conversely the consumer blocks on the buffer being empty until the producer pushes new messages (signals **not_empty**).

2.3 Meeting Real time Specification

To achieve resilience of the embedded system for 24 hours or more a few techniques were used. To start, the application should handle disconnects from the websocket. The solution is to simply configure the websocket handler to try to reconnect every second until a new connection is established. During this time no messages are going to arrive but due to a connection error and not because of low traffic. To distinguish between the two cases, the monitor is notified of the connection status and outputs -1 in the counter fields while there is a disconnect. Moreover, an issue were no messages were arriving even though there was a connection, resulting in a burst of thousands of messages in one seconds and a disconnect. To solve this issue, the application checks if the TCP connection has been idle for too long and restarts the connection (since the traffic is supposed to be rather constant).

Another issue worth looking into is what happens during a power outage. Unfortunately, in this case, the application will definitely remain inactive until there is power. However, it is important to at least guaranty its proper automatic continuation during the reboot. For this reason, the app is established as a system service that starts on boot-up so the OS handles it. It will read the desired start and end times from a schedule.txt file and decide whether to log data.

Wherever dynamic memory allocation is used, like in the creation of a connection to the websocket, proper cleanup is always ensured through destroy functions and free() in the main thread.

3 Results

Here are the required graphs from the data collected during the 24 hours of 25/9.

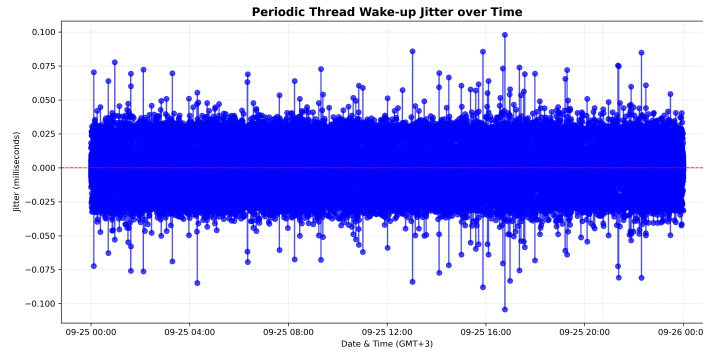


Figure 2: Jitter of monitor thread

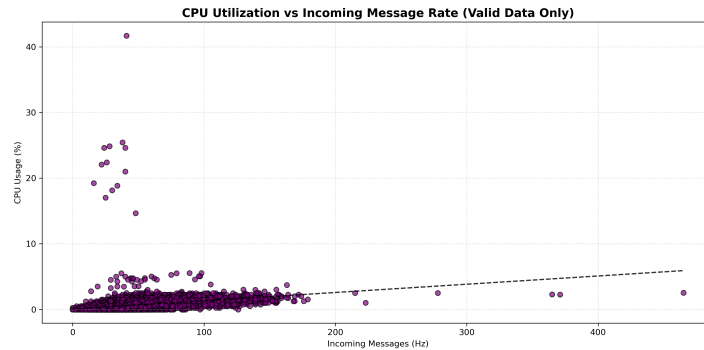


Figure 3: CPU usage vs frequency of messages

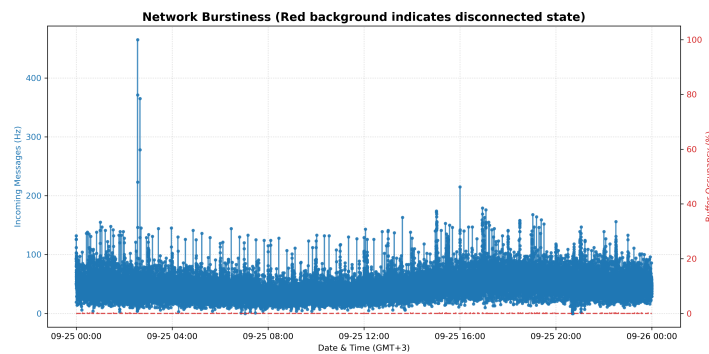


Figure 4: Frequency of messages vs time

The main aim was to minimize jitter and drift in the monitor thread, which is achieved since it is kept below 0.1 ms, a very reasonable accuracy. Furthermore, CPU utilization seems to generally stay below 6% apart from some outliers that are over 10% (probably OS internal processes or reconnection attempts). Finally, the network burstiness is shown with more or less the expected traffic. The buffer occupancy seems to stay near zero, which means there is no blocking due to the consumer being considerably slower than the producer. It should be noted that the buffer occupancy is measured every second, during which many messages have arrived and been categorized.